



Angular desde cero



Angular



**Deliver web apps
with confidence**

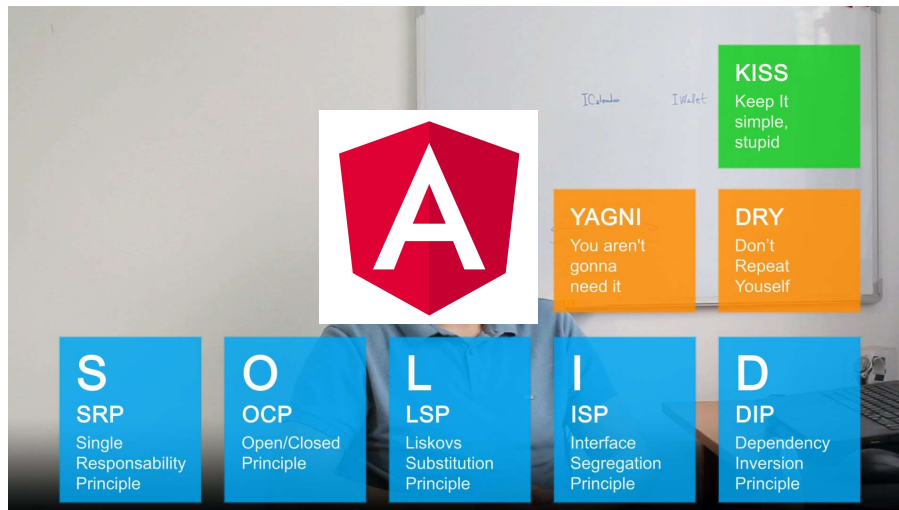


Vendiendo Angular



Angular is a platform that makes it **easy** to build ~~web applications~~ **applications with the web**

(<https://angular.io/docs>)



Angular combines (...) **best practices** to solve development challenges

(<https://angular.io/docs>)



Requisitos:

Node.js.

Opcionales: IDE

**(WebStorm, VisualStudio
code, Notepad)**



```
> npm install -g @angular/cli  
> ng new my-dream-app  
> cd my-dream-app  
> ng serve
```



The Angular CLI makes it easy to create an application that **already works**, right out of the box. It already follows **our best practices!**

(<https://angular.io/docs>)



- Crear un nuevo proyecto con angular cli
- Hacer **npm install**
- Mirar cuánto ocupa el proyecto incluyendo node_modules.

176KB vs 202 MB



Actualización automática

- Lanzar aplicación:

ng serve

- Abrir en Chrome:

<http://localhost:4200/>

- Cambiar “bienvenido a” por “welcome to”

Se actualiza automáticamente





Package.json

```
{
  "name": "formacion-ui",
  "version": "0.0.0",
  "license": "MIT",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "test": "ng test -sm=false",
    ...
  },
  "private": true,
  "dependencies": {
    "@angular/animations": "^4.2.4",
    "@angular/common": "^4.2.4",
    ...
    "zone.js": "^0.8.14"
  },
  "devDependencies": {
    "@angular/cli": "1.4.9",
    "jasmine-core": "~2.6.2",
    "karma": "~1.7.0",
    "typescript": "~2.3.3"
  }
}
```

Información general

Scripts (= atajos)

Dependencias

Dependencias de desarrollo



Scripts



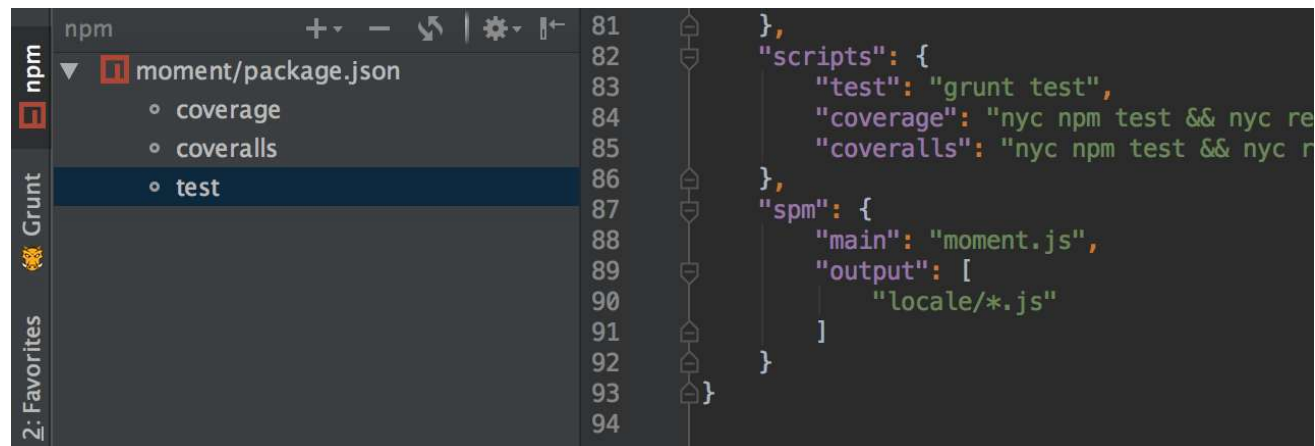
```
C:\Aplicaciones\formacion2\formacion-ui>npm run start  
  
> formacion-ui@0.0.0 start C:\Aplicaciones\formacion2\formacion-ui  
> ng serve
```



- *Útil para comandos “chorizo”.*

```
"electron-packager": "npm run prepareElectron && electron-packager ./electronTemp/ --overwrite --out=deploy && npm r
```

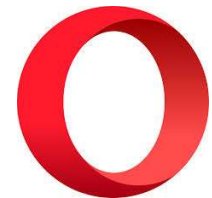
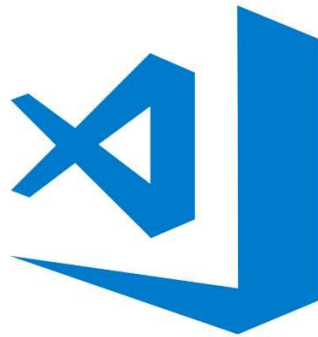
- *Integrado con los IDEs.*



```
81 },
82 "scripts": {
83   "test": "grunt test",
84   "coverage": "nyc npm test && nyc re
85   "coveralls": "nyc npm test && nyc r
86 },
87 "spm": {
88   "main": "moment.js",
89   "output": [
90     "locale/*.js"
91   ]
92 }
93 }
94 }
```



Herramientas desarrollo



Componentes





Componentes



- *Bonitas.*
- *Huecas por dentro.*
- *Hay unas dentro de otras*



- Contienen la “**interfaz**” de la aplicación.
- **No se encargan de la lógica** de la aplicación.
- Su única lógica debe ser la propia de la interfaz.
- Unos se contienen a otros.

Ejemplos:

- Mostrar o no un botón en función de los permisos.
- ~~Hacer peticiones a servidor.~~
- ~~Realizar cálculos complejos para mostrar la información.~~

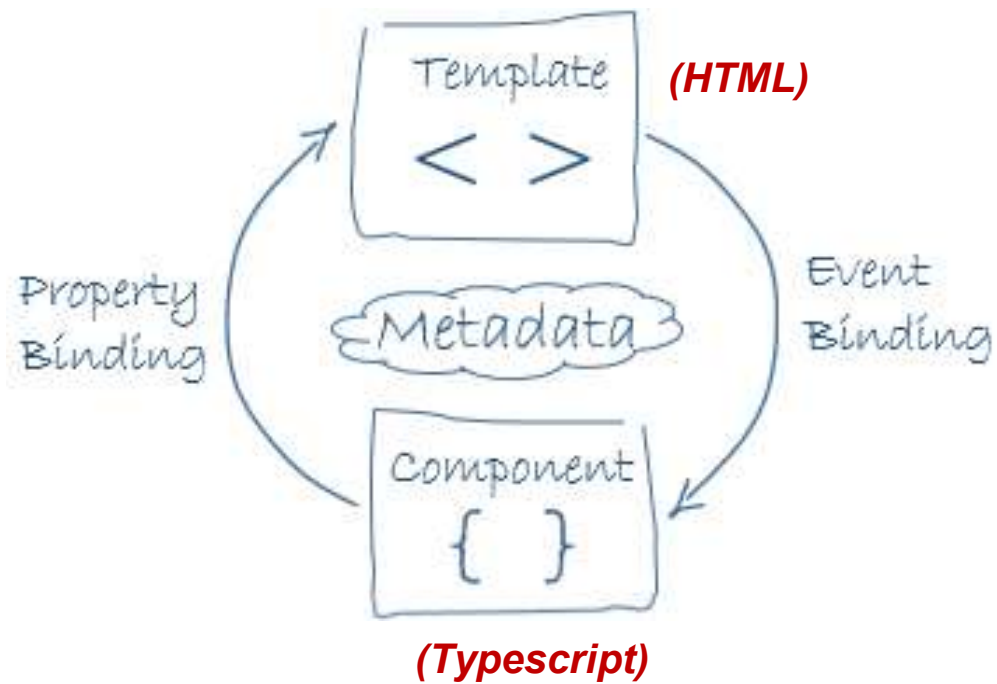


Componentes





Componentes



Typescript (muy breve)

Estructura de componente (ts)

Propiedades
públicas
(implícitamente)
Tipado explícito o
implícito

Constructor

Método público
(implícitamente)

```
import { Component } from '@angular/core';
import { User } from "../user";

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  user: User;
  title = 'app';

  constructor() {
    this.user = new User();
    this.user.nombre = "Paco";
    this.user.apellido = "Lopez";
    this.user.tipo = "Administrador";
    this.user.coches = ["Ferrari", "Subaru"];
  }

  isAdmin() {
    return this.user.tipo == "Administrador";
  }
}
```

Selector: etiqueta
para incrustarlo
dentro de la template
del componente
padre

templateUrl:
Dirección de la
template (de su
HTML)

styleUrls: Conjunto
de css a aplicar a
ese componente

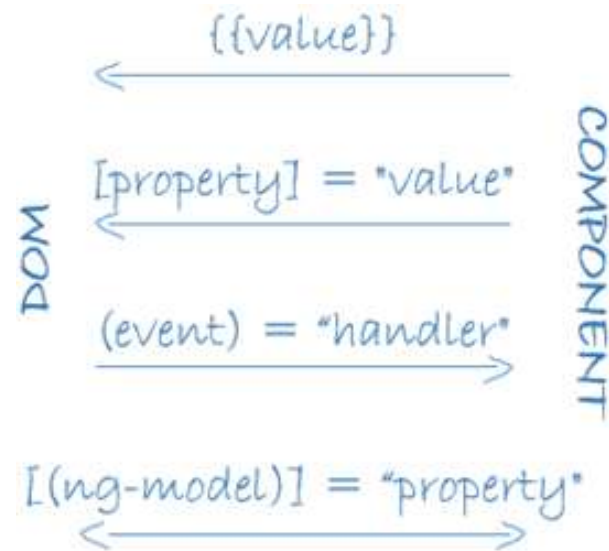


Binding

“Los bindings son el nexo entre la template (HTML) y la clase (Typescript) del componente”.



Binding





Binding {}

Mostar el usuario conectado actualmente.

⋮

{{user.nombre}}



Binding {}

```
<div style="text-align:center">
  <h1>
    Bienvenido a {{title}}! El usuario conectado actualmente es: {{user.nombre}}
  </h1>
  
</div>
<div *ngIf="isAdmin()">
  Información secreta.
</div>
<div>
  Coches:
  <ul>
    <li *ngFor="let coche of user.coches">{{coche}}</li>
  </ul>
</div>
```



⋮

“Las directivas modifican el DOM en función de una determinada condición”.



ngIf

Mostrar una información sólo si el usuario es administrador.

:

```
<div *ngIf="isAdmin()">
```



```
@if (isAdmin()){  
  <div></div>  
}
```



Binding {}

```
<div style="text-align:center">
  <h1>
    Bienvenido a {{title}}! El usuario conectado actualmente es: {{user.nombre}}
  </h1>
  
</div>
<div *ngIf="isAdmin()">
  Información secreta.
</div>
<div>
  Coches:
  <ul>
    <li *ngFor="let coche of user.coches">{{coche}}</li>
  </ul>
</div>
```



ngIf

Ejercicio: Mostrar una información sólo si el usuario es Paco.

:

OJO:

- Recordar hacer npm install
- Para abrir la aplicación hay que abrir la carpeta donde está el package.json
- Por defecto es el puerto 4200. Para cambiarlo:
<https://stackoverflow.com/questions/37762125/set-default-host-and-port-for-ng-serve-in-config-file>



ngFor

Mostrar la lista de coches del usuario.

```
<li *ngFor="let coche of user.coches">{{coche.marca}}</li>
```



```
@for (coche of user.coches; track coche.marca) {  
  <li>{{ coche.marca }}</li>  
} @empty {  
  <li>There are no items.</li>  
}
```



Binding {}

```
<div style="text-align:center">
  <h1>
    Bienvenido a {{title}}! El usuario conectado actualmente es: {{user.nombre}}
  </h1>
  
</div>
<div *ngIf="isAdmin()">
  Información secreta.
</div>
<div>
  Coches:
  <ul>
    <li *ngFor="let coche of user.coches">{{coche}}</li>
  </ul>
</div>
```



()

Poder añadir un nuevo coche a la lista.

:

:

```
<button (click)="comprarCoche()">  
Comprar coche  
</button>
```

Alternativa: `<button on-click="onSave()">On Save</button>`



No poder añadir nuevos coches si se acaba el dinero.

⋮

⋮

```
<button [disabled]="dinero < 30"  
(click)="comprarCoche()">  
  Comprar coche  
</button>
```



Más ejemplos

```
<div class="bad curly special" [class]="badCurly">Bad curly</div>
```

```
<div [class.special]="isSpecial">The class binding is special</div>
```

```
<button [style.color]="isSpecial ? 'red': 'green'">Red</button>
```

```
<tr><td [attr.colspan]="1 + 1">One-Two</td></tr>
```

```
<tr><td [colspan]="1 + 1">One-Two</td></tr> (equivalente)
```




[0]

Poder elegir el nombre del coche a añadir (por defecto Seat)



```
<input  
[(ngModel)]="nuevoCoche">
```

003_Bindings: Uso de bindings 



Módulos

```
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [  
    BrowserModule,  
    FormsModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

Componentes de este módulo

Necesidades externas del módulo

Servicios del módulo (lo vemos después)

Componentes de inicio (sólo en AppModule)



Ejercicio:

Vas a crear una aplicación web simple para administrar una lista de tareas. La aplicación permitirá al usuario agregar nuevas tareas, marcar las tareas como completadas y eliminar tareas de la lista.

Requisitos:

Vista Principal:

- ☐ Mostrar un título para la lista de tareas.
- ☐ Mostrar un formulario con un campo de entrada de texto y un botón para agregar nuevas tareas.
- ☐ Mostrar la lista de tareas existentes.

Agregar Tareas:

- ☐ Cuando el usuario escriba una nueva tarea en el campo de entrada y presione el botón de agregar, la tarea debe agregarse a la lista de tareas.
- ☐ El campo de entrada de texto debe vaciarse después de agregar una tarea.

Mostrar Tareas:

- ☐ Cada tarea debe mostrarse como un elemento de lista.
- ☐ Cada tarea debe tener una casilla de verificación para marcarla como completada.
- ☐ Las tareas completadas deben tener una apariencia diferente (por ejemplo, tachadas o con un color diferente).

Eliminar Tareas:

- ☐ Junto a cada tarea, debe haber un botón de eliminar que permita al usuario eliminar esa tarea de la lista.



Notas Adicionales:

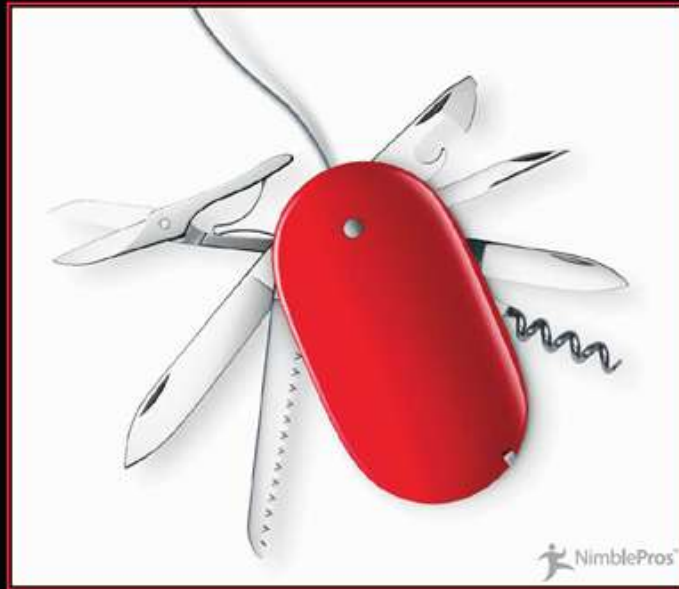
- ❑ Utiliza las directivas `*ngIf` y `*ngFor` para mostrar u ocultar elementos y para iterar sobre la lista de tareas.
- ❑ Utiliza el binding de eventos (click) para manejar la lógica de agregar y eliminar tareas.
- ❑ Utiliza el binding de propiedades `[(ngModel)]` para vincular el valor del campo de entrada de texto a una propiedad en el componente de Angular.

Componentes





Buenas prácticas



SINGLE RESPONSIBILITY

Avoid tightly coupling your tools together.

El componente hace demasiadas cosas:

- ☐ *Añadir una nueva tarea*
- ☐ *Eliminar las tareas*
- ☐ *Mostrar las tareas.*



Nuevo componente

```
<h1>Lista de tarea</h1>

<div>
  <mat-form-field class="example-full-width">
    <mat-label>Tarea</mat-label>
    <input type="text" [(ngModel)] = "nuevaTarea" id="NombreTarea" matInput
    | | | | | | | | | | placeholder="Escribe tus tareas"><br>
  </mat-form-field>
  <button mat-flat-button color="primary" (click)="AniadirTarea()">Añadir</button>
</div>

<br>

<mat-selection-list >
  @for (tarea of listaTareas; track tarea) {
    <mat-list-option [(selected)]="tarea.terminado">
      <span [style.text-decoration]="tarea.terminado ? 'line-through' : 'none'">
        {{tarea.nombre}}</span>
      <button mat-flat-button color="warn" (click)="Eliminartarea(tarea)">Eliminar</button>
    </mat-list-option>
  }
</mat-selection-list>
```

→ ListaTareas
Component



Angular CLI

—
:

—
:

ng generate component ListaTareas
Abreviado: ng g c ListaTareas



Se crean los siguientes archivos:



- lista-tareas.component.html: template (HTML)
- lista-tareas.component.ts: clase (Typescript)
- lista-tareas.component.spec.ts: Pruebas.
- lista-tareas.component.css: Estilos (css/sass/less)



Metadatos

lista-tareas.component.ts

```
@Component({  
  selector: 'app-lista-tareas',  
  templateUrl: './lista-tareas.component.html',  
  styleUrls: ['./lista-tareas.component.css']  
})  
export class ListaTareasComponent implements OnInit {
```

app.component.html

```
<app-lista-tareas></app-lista-tareas>
```



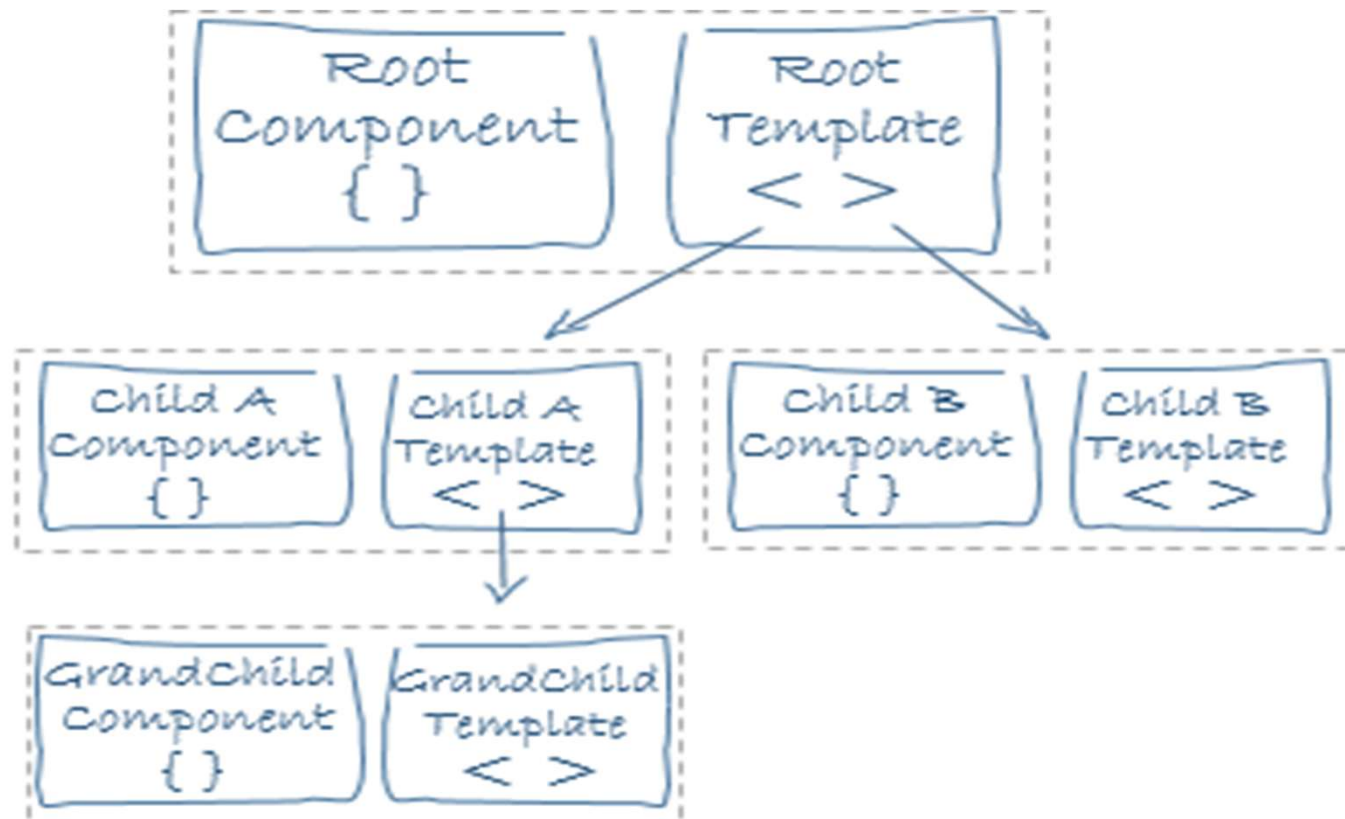
El **ciclo de vida de un componente en Angular** describe las distintas fases por las que pasa un componente desde que se crea hasta que se destruye.

.

Orden	Hook	Cuándo se llama
1	constructor()	Cuando se instancia el componente (antes de inicializar inputs o la vista).
2	ngOnChanges(changes)	Cada vez que se cambian los inputs (también se llama la primera vez).
3	ngOnInit()	Una vez después del primer ngOnChanges; ideal para inicializaciones.
4	ngAfterViewInit()	Cuando la vista del componente y sus hijos están inicializados.
5	ngOnDestroy()	Justo antes de destruir el componente; se usa para limpiar suscripciones, etc.



Componentes





Ejercicio

Crear el componente personalizado listaTareas y añadirlo a AppComponent.

Debe tener:

- *Algún binding unidireccional: {}.*
- *Algún *ngIf y *ngFor.*
- *Algún evento: ().*
- *Algún property binding: [].*
- *Algún binding bidireccional: [()]*

<https://angular.io/guide/template-syntax>



Componentes





¿Huecas por dentro?

¡No!
Hay algo dentro que no debería estar



```
AniadirTarea(){
  for(let i=0;i<this.listaTareas.length;i++){
    if(this.listaTareas[i].nombre == this.nuevaTarea){
      alert("La tarea ya existe");
      return;
    }
  }
  this.listaTareas.push({nombre:this.nuevaTarea,terminado:false});
  this.nuevaTarea = "";
}

CambiarEstado(tarea: Tarea){
  tarea.terminado = !tarea.terminado;
}

EliminarTarea(tarea: Tarea){
  this.listaTareas = this.listaTareas.filter(t => t !== tarea);
}
```

***¡¡El componente
está realizando
lógica!!***

Servicios





Servicios

Realizan la **lógica** de la aplicación:

- Facilitar la **comunicación** entre componentes.
- Realizar peticiones al **back-end**.
- Realizar **cálculos**/algoritmos más complejos (ej. D3).
- Almacenar el **estado** de la aplicación (discutible).



Llevar la lógica del componente al servicio.

-

-

***ng generate service ListaTareas
(ng g s ListaTareas)***

005_Nuestro_primer_servicio





Generate Service

Se crean los siguientes archivos:

•

- lista-tareas.service.ts. Clase del servicio
- lista-tareas.service.spec.ts. Pruebas del servicio



- *Declarar que el componente depende del servicio.*
- *Hacer que el componente use el servicio.*
- *Configurar el contenedor.*

```
@Injectable({providedIn: 'root'})  
export class HeroService {
```



1) Configuración a nivel de aplicación:

```
@NgModule({  
  ...  
  providers: [CarService],  
  ...  
})  
export class AppModule { }
```

```
import { Injectable } from '@angular/core';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class SquareService {  
  
  constructor() { }  
  
  calcularCuadrado(numero: number): number {  
    return numero * numero;  
  }  
}
```

Toda la aplicación compartirá la misma instancia.

Útil para servicios generales: Ej. NestingFileService



2) Configuración a nivel de componente:

```
@Component({  
  selector: 'app-car',  
  templateUrl: './car.component.html',  
  styleUrls: ['./car.component.css'],  
  providers: [CarService]  
})  
export class CarComponent {
```

Cada componente de este tipo tendrá su propio servicio.
Sus hijos tendrán el mismo que el padre salvo que lo especifiquen explícitamente en su anotación providers
Útil para llevar lógica de componente a servicio.

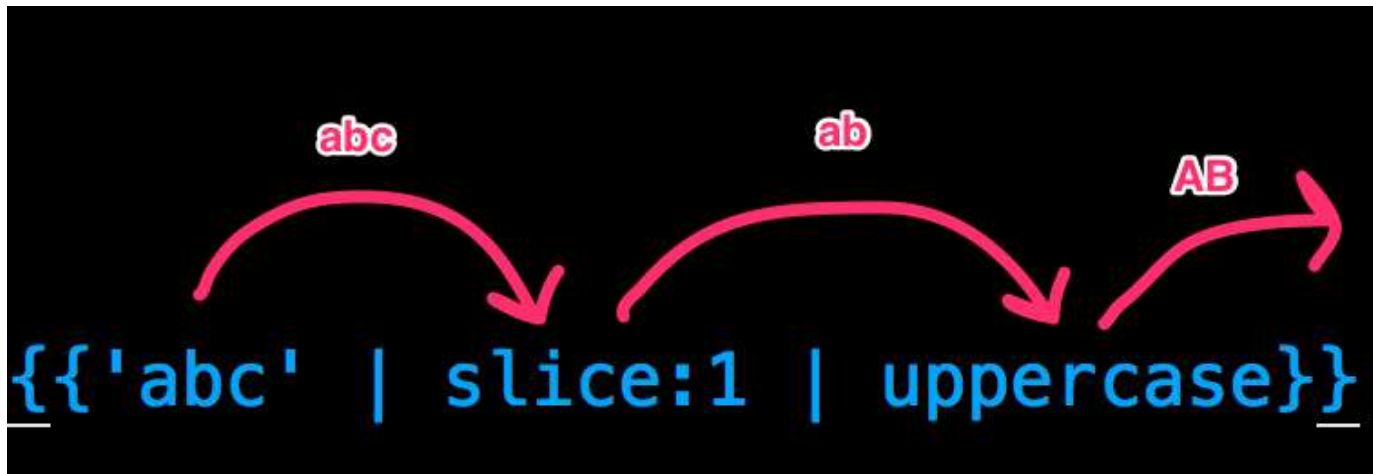
Pipes





Pipes

- ▣ *Sirven para convertir un dato en otro.*
- ▣ *Se pueden encadenar. De ahí el nombre pipe.*



- ▣ *Uso típico: Convertir dato en texto para mostrarlo.*



Pipes

- Crearla

ng generate pipe User

- Probarla

En user.pipe.spec.ts

- Implementarla

En user.pipe.ts

- Usarla:

`{{user| userDecripcion}}`



Pipes

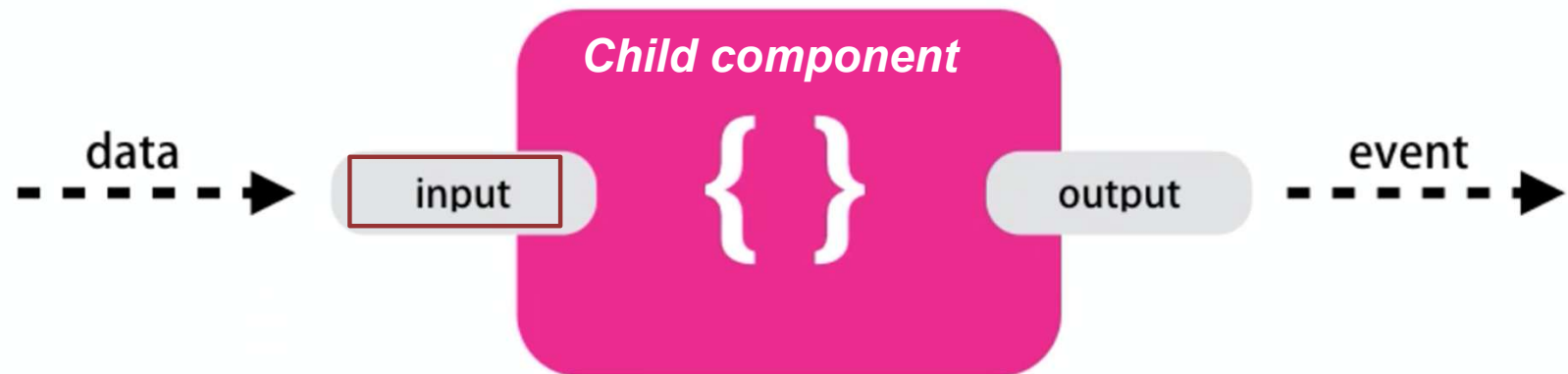
```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'uppercase'
})
export class UppercasePipe implements PipeTransform {
  transform(value: string): string {
    return value.toUpperCase();
  }
}
```

Comunicación entre componentes: Input y Output



Parent component





Input

Child component

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.css']
})
export class UserCardComponent {
  @Input() user: any;
}
```

```
<div class="user-card">
  <h2>{{ user.name }}</h2>
  <p>Email: {{ user.email }}</p>
  <p>Edad: {{ user.age }}</p>
</div>
```



Input

Parent component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  user = {
    name: 'John Doe',
    email: 'john@example.com',
    age: 30
  };
}
```

```
<app-user-card [user]="user"></app-user-card>
```



Output

Child component

```
/* child.component.html */  
<h2>Hi, I'm a nested component</h2>  
<span (click)='onClick()'>Click me please!</span>
```

```
@Component({  
  selector: 'child-selector',  
  template: 'child.component.html'  
})  
export class ChildComponent {  
  @output() notify: EventEmitter<string> = new EventEmitter<string>()  
  
  onClick() {  
    this.notify.emit('Click from nested component');  
  }  
}
```




Parent component

Output

```
/* parent.component.html */  
<div>  
  <h1>I'm a container component</h1>  
  <child-selector (notify)='onNotify($event)'></child-selector>  
</div>
```

```
@Component({  
  selector: 'parent-selector',  
  template: 'parent.component.html'  
  directives: [ChildComponent]  
})  
export class ParentComponent {  
  onNotify(message:string):void {  
    alert(message);  
  }  
}
```



Ejercicio

Haciendo uso de Inputs y outputs, cumplir con el principio de responsabilidad única para refactorizar la aplicación, teniendo dos componentes nuevos:

- *FormularioComponent, que lista los coches.*
- *ListaComponent, que permite comprar un coche.*

Directivas personalizadas





- Permite extender o modificar el comportamiento de los elementos del DOM
- Pueden aplicar lógica de presentación o comportamiento

```
ng generate directive highlight
```



Directivas

```
import { Directive, ElementRef, HostListener, Input } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @Input('appHighlight') highlightColor: string;

  constructor(private el: ElementRef) { }

  @HostListener('mouseenter') onMouseEnter() {
    this.highlight(this.highlightColor || 'yellow');
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.highlight(null);
  }

  private highlight(color: string) {
    this.el.nativeElement.style.backgroundColor = color;
  }
}
```



Directivas

```
<p appHighlight highlightColor="yellow">¡Pasa el mouse sobre mí para resaltarme!</p>
```

Comunicación con back-end





Llamada a HttpClient

```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  private apiUrl = 'https://api.example.com/users'; // URL de la API

  constructor(private http: HttpClient) { }

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }
}
```




Subscripción al observable

```
@Component({
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
export class UserComponent implements OnInit {
  users: User[] = [];

  constructor(private userService: UserService) { }

  ngOnInit(): void {
    this.userService.getUsers().subscribe(users => {
      this.users = users;
    });
  }
}
```



Nuestro objetivo:

- *No tener ni que saber de esto.*
- *Cómo: automatizando.*



Estándar Open API

La funcionalidad ofrecida por cada servicio se documenta mediante el estándar Open API (JSON)
Cualquier sistema externo autenticado puede acceder a ella.



Visualizar documentación

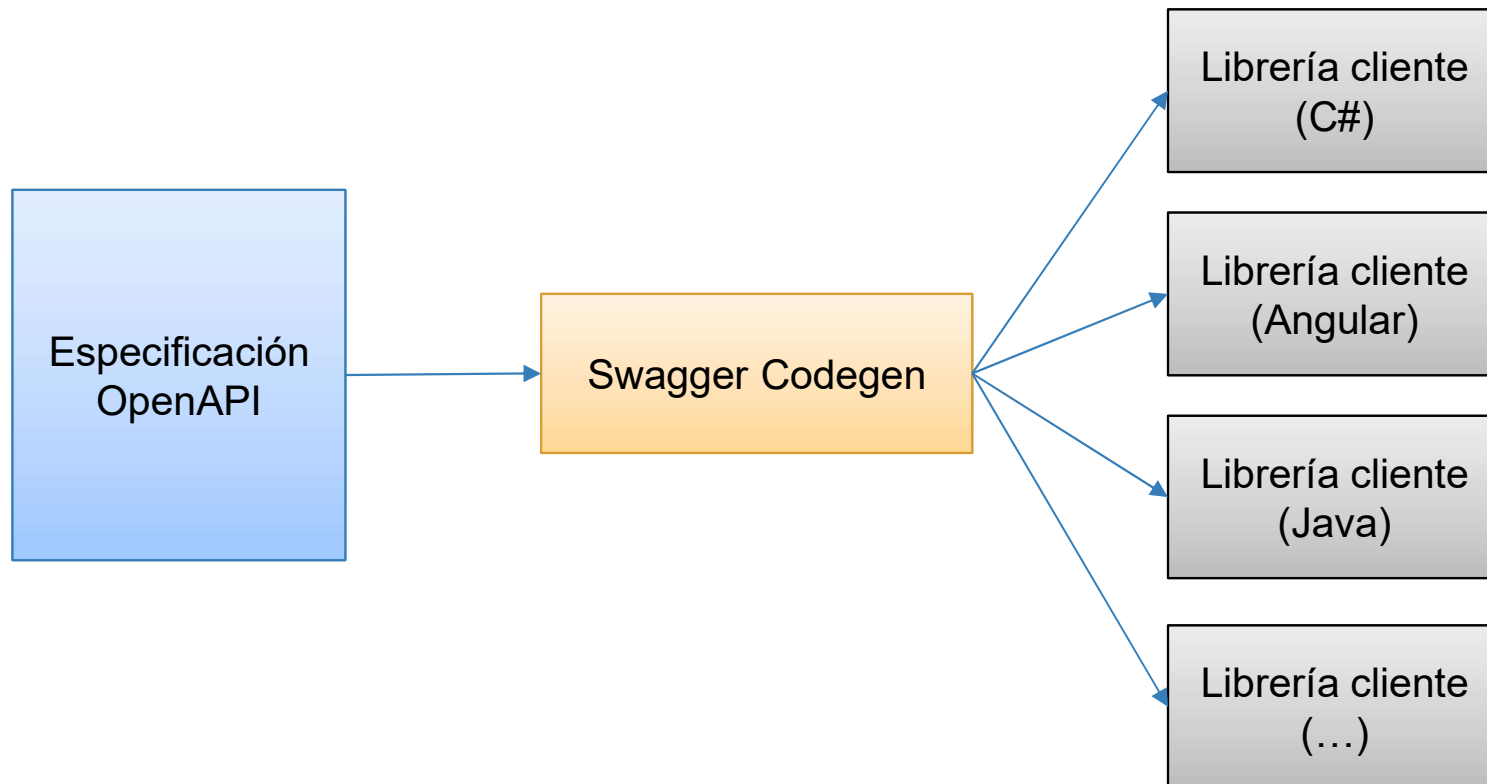
Existen herramientas para visualizar la documentación generada.

The screenshot displays the Cisco Tecoi API documentation interface. At the top, there is a yellow header with the Tecoi logo and the text "Cisco 1.0.0". Below this, a blue banner features the Tecoi logo and the text "Cisco cutting software" and "Contact Tecoi". A green "Authorize" button is located in the top right corner. The main content area is divided into three sections: "Reference", "Notifications", and "Nesting Files". Each section contains a list of API endpoints with their respective HTTP methods, descriptions, and status icons.

Method	Endpoint	Description	Status
GET	/reference	Get the reference of the Cutting Charts	Locked
PUT	/reference	Updates the reference	Locked
GET	/notifications/current	Get the machine notifications	Locked
DELETE	/notifications/current	Reset the machine notifications	Locked
PUT	/nesting-files/current	Updates a Nesting File	Locked
POST	/nesting-files/current	Adds a Nesting File	Locked
DELETE	/nesting-files/current	Deletes a Nesting File	Locked

Generar código cliente

A partir de la documentación, es posible generar **código cliente** automáticamente en distintos lenguajes. Estos clientes ***abstraen*** de toda la comunicación HTTP. La convierte en simples llamadas a métodos.





Input

Ejercicio

Haciendo uso swagger, generar el servicio que permita hacer llamadas al api .NET de gestión de tareas y realizar toda la lógica en el backend

Internacionalizacion (i18n)





i18n

- La internacionalización es el proceso por el cual preparamos nuestra aplicación para que se visualice en diferentes idiomas
- Instalación del paquete de internacionalización:

```
ng add @angular/localize
```




i18n

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "i18n-angular-lokalise": {
      "projectType": "application",
      // ...
      "i18n": { // <--- 1
        "sourceLocale": "en-US", // <--- 2
        "locales": { // <--- 3
          "ru": { // <--- 4
            "translation": "src/locale/messages.ru.xlf" // <--- 5
          }
        }
      },
      "architect": {
        "build": {
          "builder": "@angular-devkit/build-angular:browser",
          "options": {
            "localize": true, // <--- 6
            "i18nMissingTranslation": "error", // <--- 7
            // ...
          },
          "configurations": {
            // ...
          },
          "defaultConfiguration": "production"
        },
        // ...
      }
    }
  }
}
```

Modificaciones angular.json

```
"build": {
  // ...
  "configurations": {
    // ...
    "ru": {
      "localize": ["ru"] // <--- 1
    }
  },
  // ...
},
"serve": {
  "builder": "@angular-devkit/build-angular:dev-server",
  "configurations": {
    // ...
    "ru": {
      "browserTarget": "i18n-angular-lokalise:build:development,ru" // <--- 2
    }
  },
  // ...
}
```



i18n

- Marcar textos en componentes

```
<h1 i18n="@@HelloI18n">Hello i18n!</h1>
```

- Marcar atributos en componentes

```

```

- Marcar textos en código (.ts)

```
title = $localize`:@@Title:Some title text`
```



i18n

- Extracción de todos los textos de la aplicación a un fichero plantilla

```
ng extract-i18n
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="en-US" datatype="plaintext" original="ng2.template">
    <body>
      <trans-unit id="HelloI18n" datatype="html">
        <source>Hello i18n!</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">330</context>
        </context-group>
      </trans-unit>
      <trans-unit id="angularLogo" datatype="html">
        <source>Angular logo</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">332</context>
        </context-group>
      </trans-unit>
      <trans-unit id="Title" datatype="html">
        <source>Some title text</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.ts</context>
          <context context-type="linenumber">10</context>
        </context-group>
      </trans-unit>
    </body>
  </file>
</xliff>
```



i18n

- Añadimos una copia del fichero generado por cada idioma. Ejem. messages.es.xlf

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="en-US" datatype="plaintext" original="ng2.template">
    <body>
      <trans-unit id="HelloI18n" datatype="html">
        <source>Hello i18n!</source>
        <target>¡Hola i18n!</target>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">330</context>
        </context-group>
      </trans-unit>
      <trans-unit id="angularLogo" datatype="html">
        <source>Angular logo</source>
        <target>Logo de Angular</target>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">332</context>
        </context-group>
      </trans-unit>
      <trans-unit id="Title" datatype="html">
        <source>Some title text</source>
        <target>Algun texto de titulo</target>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.ts</context>
          <context context-type="linenumber">10</context>
        </context-group>
      </trans-unit>
    </body>
  </file>
</xliff>
```



i18n

- Lanzamos la aplicación en el idioma deseado

```
ng serve --configuration=es
```

Routing



- Permite navegar entre distintos componentes mediante la URL
- app.route.ts

```
const routes: Routes = [  
  { path: 'first-component', component: FirstComponent },  
  { path: 'second-component', component: SecondComponent },  
  { path: '', redirectTo: '/first-component', pathMatch: 'full' }, //  
  redirect to `first-component`  
  { path: '**', component: PageNotFoundComponent }, // Wildcard route for a  
  404 page  
];
```



□ App.component.html

```
<h1>Angular Router App</h1>
<nav>
  <ul>
    <li><a routerLink="/first-component" routerLinkActive="active"
ariaCurrentWhenActive="page">First Component</a></li>
    <li><a routerLink="/second-component" routerLinkActive="active"
ariaCurrentWhenActive="page">Second Component</a></li>
  </ul>
</nav>
<!-- The routed views render in the <router-outlet>-->
<router-outlet></router-outlet>
```




□ App.component.ts

```
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [CommonModule, RouterOutlet, RouterLink, RouterLinkActive],  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  title = 'routing-app';  
}
```